

IFRN — Instituto Federal do Rio Grande do Norte

Relacionamentos entre Modelos

Django REST Framework

Prof. Plácido Neto

Objetivos desta Aula

Ao final desta aula\ o aluno será capaz de:

- Compreender os três tipos fundamentais de relacionamento no ORM do Django: ForeignKey (1:N), OneToOneField (1:1) e ManyToManyField (N:N);
- Modelar entidades associativas (tabelas de junção) com atributos próprios, como o ItemPedido;
- Configurar o parâmetro on_delete e o atributo related_name para controlar o comportamento de exclusão e a navegação reversa entre objetos;
- Implementar serializers aninhados (nested serializers) e usar PrimaryKeyRelatedField, StringRelatedField e SerializerMethodField;
- Customizar os métodos create() e update() do serializer para tratar inserções com objetos relacionados em uma única requisição;
- Validar a integridade dos endpoints com Swagger UI, REST Client e Insomnia/Postman.

1. Revisão: O Cenário Atual da Amazon API

Nas aulas anteriores construímos três modelos completamente independentes: Cliente, Vendedor e Produto. Cada um com seu próprio CRUD, serializer e ViewSet. Em um sistema real, porém, essas entidades raramente vivem isoladas: clientes fazem pedidos, pedidos contêm produtos, e produtos pertencem a vendedores. Esses vínculos são chamados de relacionamentos e formam o esqueleto de qualquer modelo de dados realista.

Nesta aula vamos transformar a Amazon API em um modelo coeso, criando três novas entidades que conectam todas as anteriores:

- PerfilVendedor — relacionamento 1:1 (OneToOne) com Vendedor, armazenando dados complementares;
- Pedido — relacionamento N:1 (ForeignKey) com Cliente, representando uma compra;
- ItemPedido — entidade associativa N:N (ManyToMany resolvida explicitamente) entre Pedido e Produto, com atributos próprios (quantidade, preço unitário).

2. Conceitos Fundamentais de Relacionamento

Antes de escrever código, é essencial dominar três tipos de relacionamento no ORM do Django. Cada um expressa uma cardinalidade específica entre tabelas e gera um SQL diferente.

Tipo	Campo Django	SQL Gerado	Exemplo
1:N (Um para Muitos)	ForeignKey	Coluna FK em tabela filha	Um Cliente faz N Pedidos
1:1 (Um para Um)	OneToOneField	FK + UNIQUE constraint	Um Vendedor tem um PerfilVendedor

Tipo	Campo Django	SQL Gerado	Exemplo
N:N (Muitos para Muitos)	ManyToManyField	Tabela associativa intermediária	Pedidos contêm vários Produtos

Tabela 19.1 — Tipos de relacionamento no Django ORM

2.1 Parâmetros Essenciais de um Relacionamento

Todos os campos de relacionamento aceitam dois parâmetros que merecem atenção especial: `on_delete` e `related_name`.

2.1.1 `on_delete` — O que fazer ao excluir o registro pai?

Quando o objeto referenciado por uma `ForeignKey` é apagado, o Django precisa saber o que fazer com os filhos. As opções mais usadas são:

Opção	Comportamento	Quando usar
CASCADE	Apaga também os filhos	Itens de pedido (sem pedido, item não faz sentido)
PROTECT	Impede a exclusão do pai (<code>ProtectedError</code>)	Cliente com histórico de compras
SET_NULL	Define o campo FK como NULL (exige <code>null=True</code>)	Produto sem vendedor ativo
SET_DEFAULT	Define o valor padrão definido em <code>default=</code>	FK com valor padrão lógico
DO_NOTHING	Não faz nada (perigoso, viola integridade)	Quase nunca é recomendado

Tabela 19.2 — Opções do parâmetro `on_delete`

2.1.2 `related_name` — Como acessar os filhos a partir do pai?

Por padrão, o Django cria um acessor reverso chamado `<modelo_filho>_set`. Por exemplo, sem `related_name`, para listar os pedidos de um cliente seria necessário: `cliente.pedido_set.all()`. Definir `related_name='pedidos'` permite a sintaxe muito mais natural `cliente.pedidos.all()`.

 **Boa prática:** Sempre defina `related_name` no plural, em português, e no contexto do pai. Isso torna o código de consultas reversas autoexplicativo e elimina ambiguidades quando o mesmo modelo é referenciado por múltiplas FKs.

3. Relacionamento 1:1 — Modelo PerfilVendedor

O relacionamento 1:1 é usado para estender um modelo existente sem inflar a tabela original. No nosso caso, queremos guardar dados bancários e fiscais do vendedor (informações sensíveis e usadas com menos frequência) em uma tabela separada, mantendo `Vendedor` enxuto.

3.1 Definição do Modelo

```
# backend/models.py (adicionar após a classe Produto)
class PerfilVendedor(models.Model):
```

```

vendedor = models.OneToOneField(
    Vendedor,
    on_delete=models.CASCADE,
    related_name='perfil',
    primary_key=True # usa o id do vendedor como PK
)
razao_social = models.CharField(max_length=150, blank=True)
inscricao_estadual = models.CharField(max_length=20, blank=True)
banco = models.CharField(max_length=50, blank=True)
agencia = models.CharField(max_length=10, blank=True)
conta = models.CharField(max_length=20, blank=True)
chave_pix = models.CharField(max_length=100, blank=True)

class Meta:
    db_table = 'perfis_vendedores'

def __str__(self):
    return f'Perfil de {self.vendedor.nome}'

```

Código 20.1 — Modelo PerfilVendedor com OneToOneField

 **Atenção:** O parâmetro `primary_key=True` faz o id do Vendedor servir como chave primária do PerfilVendedor. Isso garante que cada vendedor tenha no máximo um perfil sem precisar de uma coluna id adicional.

3.2 Acessando o Relacionamento Reverso

Com `related_name='perfil'`, podemos navegar nos dois sentidos:

```

# Sentido direto (do filho para o pai)
perfil = PerfilVendedor.objects.get(pk=1)
print(perfil.vendedor.nome) # acessa o Vendedor

# Sentido reverso (do pai para o filho)
vendedor = Vendedor.objects.get(pk=1)
print(vendedor.perfil.chave_pix) # acessa o PerfilVendedor

```

Código 20.2 — Navegação bidirecional no relacionamento 1:1

4. Relacionamento 1:N — Modelo Pedido

Um Cliente pode realizar muitos Pedidos, mas cada Pedido pertence a exatamente um Cliente. Esse é o relacionamento mais comum em aplicações reais e é representado por um campo `ForeignKey` no lado N (filho).

21.1 Definição do Modelo

```

# backend/models.py (adicionar após PerfilVendedor)
class Pedido(models.Model):
    STATUS_CHOICES = [
        ('pendente', 'Pendente'),
        ('pago', 'Pago'),
        ('enviado', 'Enviado'),
        ('entregue', 'Entregue'),
        ('cancelado', 'Cancelado'),
    ]

    cliente = models.ForeignKey(
        Cliente,
        on_delete=models.PROTECT, # cliente com pedidos não pode ser apagado
    )

```

```

        related_name='pedidos'
    )
    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default='pendente'
    )
    data_pedido = models.DateTimeField(auto_now_add=True)
    observacoes = models.TextField(blank=True)

    class Meta:
        db_table = 'pedidos'
        ordering = ['-data_pedido']

    def __str__(self):
        return f'Pedido #{self.id} - {self.cliente.nome}'

```

Código 21.1 — Modelo Pedido com ForeignKey para Cliente

 **@property:** Um método decorado com @property pode ser acessado como atributo: pedido.total (sem parênteses). É ideal para campos calculados a partir de relacionamentos, como a soma dos itens de um pedido.

4.2 Por que PROTECT em vez de CASCADE?

Imagine um cenário real: se um cliente é apagado por engano e o relacionamento estiver em CASCADE, todos os pedidos dele desaparecem junto, levando com eles o histórico financeiro e fiscal da empresa. Usar PROTECT força o desenvolvedor a tomar uma decisão consciente: ou cancela os pedidos antes, ou mantém o cliente.

5. Relacionamento N:N com Atributos — ItemPedido

Um Pedido contém vários Produtos, e o mesmo Produto pode aparecer em vários Pedidos. Essa é uma relação muitos-para-muitos clássica. Porém, precisamos guardar a quantidade comprada e o preço congelado no momento da compra (afinal, o preço do produto pode mudar). Isso só é possível com uma tabela associativa explícita.

 **Conceito-chave:** Sempre que a relação N:N tiver atributos próprios (data, quantidade, preço, status, etc.), criamos um modelo intermediário com duas ForeignKeys e os campos extras. Não usamos ManyToManyField puro nesse caso.

5.1 Definição do Modelo

```

# backend/models.py (adicionar após Pedido)
class ItemPedido(models.Model):
    pedido = models.ForeignKey(
        Pedido,
        on_delete=models.CASCADE,      # itens não existem sem o pedido
        related_name='itens'
    )
    produto = models.ForeignKey(
        Produto,
        on_delete=models.PROTECT,      # produto com vendas não pode ser apagado
        related_name='itens_vendidos'
    )
    quantidade = models.PositiveIntegerField(default=1)

```

```

preco_unitario = models.DecimalField(
    max_digits=10,
    decimal_places=2,
    help_text='Preço congelado no momento da compra'
)

class Meta:
    db_table = 'itens_pedido'
    unique_together = ['pedido', 'produto'] # mesmo produto não duplica no
pedido

def __str__(self):
    return f'{self.quantidade}x {self.produto.nome}'

@property
def subtotal(self):
    return self.quantidade * self.preco_unitario

```

Código 22.1 — Modelo associativo ItemPedido

💡 **@property:** Um método decorado com @property pode ser acessado como atributo: pedido.total (sem parênteses). É ideal para campos calculados a partir de relacionamentos, como a soma dos itens de um pedido.

Campo	Tipo Django	Observação
pedido	ForeignKey(Pedido)	CASCADE — apagar pedido apaga itens
produto	ForeignKey(Produto)	PROTECT — preserva histórico
quantidade	PositiveIntegerField	Apenas inteiros >= 0
preco_unitario	DecimalField(10,2)	Snapshot do preço na compra
subtotal	@property	Calculado: quantidade × preço

Tabela 22.1 — Estrutura do modelo ItemPedido

6. Aplicando as Migrações

```

# Gerar arquivos de migração para os 3 novos modelos
python manage.py makemigrations

# Saída esperada:
# Migrations for 'backend':
#   backend/migrations/0003_pedido_perfilvendedor_itempedido.py
#       + Create model PerfilVendedor
#       + Create model Pedido
#       + Create model ItemPedido

# Aplicar as migrações ao banco
python manage.py migrate

# Conferir o SQL que será executado (sem aplicar)
python manage.py sqlmigrate backend 0003

```

 **Ordem importa:** O Django detecta automaticamente as dependências entre modelos e cria as tabelas na ordem correta: primeiro os modelos referenciados (Cliente, Vendedor, Produto) e depois os que os referenciam (Pedido, ItemPedido, PerfilVendedor).

7. Serializers com Relacionamentos

Em aulas anteriores, todos os serializers usavam `fields = '__all__'` e a serialização era direta. Quando há relacionamentos, surgem novas decisões: devemos retornar apenas o ID do objeto relacionado, ou o objeto completo aninhado? E ao criar um pedido, como inserir vários itens em uma só requisição?

7.1 Os Quatro Modos de Representar uma Relação

Estratégia	Tipo de Campo	JSON Resultante
ID apenas (padrão)	PrimaryKeyRelatedField	"cliente": 1
String <code>__str__()</code>	StringRelatedField	"cliente": "Maria da Silva"
Hyperlink	HyperlinkedRelatedField	"cliente": "/api/clientes/1/"
Objeto completo	Serializer aninhado	"cliente": { "id": 1, "nome": "..."} }

Tabela 24.1 — Estratégias de serialização de relacionamentos

7.2 Serializer com Relacionamento 1:1 — PerfilVendedor

```
# backend/serializers.py
from rest_framework import serializers
from .models import (Cliente, Vendedor, Produto,
                    PerfilVendedor, Pedido, ItemPedido)

class PerfilVendedorSerializer(serializers.ModelSerializer):
    vendedor_nome = serializers.CharField(
        source='vendedor.nome',
        read_only=True
    )

    class Meta:
        model = PerfilVendedor
        fields = ['vendedor', 'vendedor_nome', 'razao_social',
                'inscricao_estadual', 'banco', 'agencia',
                'conta', 'chave_pix']
```

Código 24.1 — Serializer para o relacionamento 1:1

7.3 Serializer Aninhado para ItemPedido

O `ItemPedidoSerializer` mostra dois recursos importantes: o uso de `SerializerMethodField` para campos calculados e o de `source` para acessar atributos de objetos relacionados.

```
class ItemPedidoSerializer(serializers.ModelSerializer):
    produto_nome = serializers.CharField(
        source='produto.nome',
        read_only=True
    )
    subtotal = serializers.SerializerMethodField()
```

```
class Meta:
    model = ItemPedido
    fields = ['id', 'produto', 'produto_nome',
              'quantidade', 'preco_unitario', 'subtotal']

def get_subtotal(self, obj):
    return obj.quantidade * obj.preco_unitario
```

Código 24.2 — ItemPedidoSerializer com campos calculados

7.4 Serializer Aninhado para Pedido (write/read)

Aqui está o ponto mais delicado: queremos receber os itens junto com o pedido em um único POST e, ao mesmo tempo, retornar os itens completos quando o cliente fizer um GET. A solução é sobrescrever os métodos `create()` e `update()` do serializer.

```
class PedidoSerializer(serializers.ModelSerializer):
    itens = ItemPedidoSerializer(many=True)
    cliente_nome = serializers.CharField(
        source='cliente.nome',
        read_only=True
    )
    total = serializers.SerializerMethodField()

    class Meta:
        model = Pedido
        fields = ['id', 'cliente', 'cliente_nome', 'status',
                  'data_pedido', 'observacoes', 'itens', 'total']
        read_only_fields = ['data_pedido']

    def get_total(self, obj):
        return sum(i.quantidade * i.preco_unitario
                   for i in obj.itens.all())

    def create(self, validated_data):
        itens_data = validated_data.pop('itens')
        pedido = Pedido.objects.create(**validated_data)
        for item_data in itens_data:
            ItemPedido.objects.create(pedido=pedido, **item_data)
        return pedido

    def update(self, instance, validated_data):
        itens_data = validated_data.pop('itens', None)
        # Atualiza campos do pedido
        for attr, value in validated_data.items():
            setattr(instance, attr, value)
        instance.save()
        # Substitui completamente os itens, se enviados
        if itens_data is not None:
            instance.itens.all().delete()
            for item_data in itens_data:
                ItemPedido.objects.create(pedido=instance, **item_data)
        return instance
```

Código 24.3 — PedidoSerializer com create/update aninhados



Por que sobrescrever `create()`? O `ModelSerializer` padrão do DRF não sabe lidar com objetos aninhados em escrita: ele chamaria `Pedido.objects.create(itens=[...])`, o que falharia.

Por isso, retiramos os itens antes (pop), criamos o pai e, em seguida, criamos cada filho associando-o ao pai recém-criado.

8. ViewSets e Otimização de Consultas

8.1 Implementação dos ViewSets

```
# backend/views.py (adicionar imports e classes)
from .models import (Cliente, Vendedor, Produto,
                    PerfilVendedor, Pedido, ItemPedido)
from .serializers import (ClienteSerializer, VendedorSerializer,
                        ProdutoSerializer, PerfilVendedorSerializer,
                        PedidoSerializer, ItemPedidoSerializer)

class PerfilVendedorViewSet(viewsets.ModelViewSet):
    queryset = PerfilVendedor.objects.select_related('vendedor').all()
    serializer_class = PerfilVendedorSerializer

class PedidoViewSet(viewsets.ModelViewSet):
    serializer_class = PedidoSerializer

    def get_queryset(self):
        return (Pedido.objects
                .select_related('cliente')
                .prefetch_related('itens__produto')
                .all())

class ItemPedidoViewSet(viewsets.ModelViewSet):
    queryset = ItemPedido.objects.select_related('pedido', 'produto').all()
    serializer_class = ItemPedidoSerializer
```

Código 25.1 — ViewSets com otimização de consultas

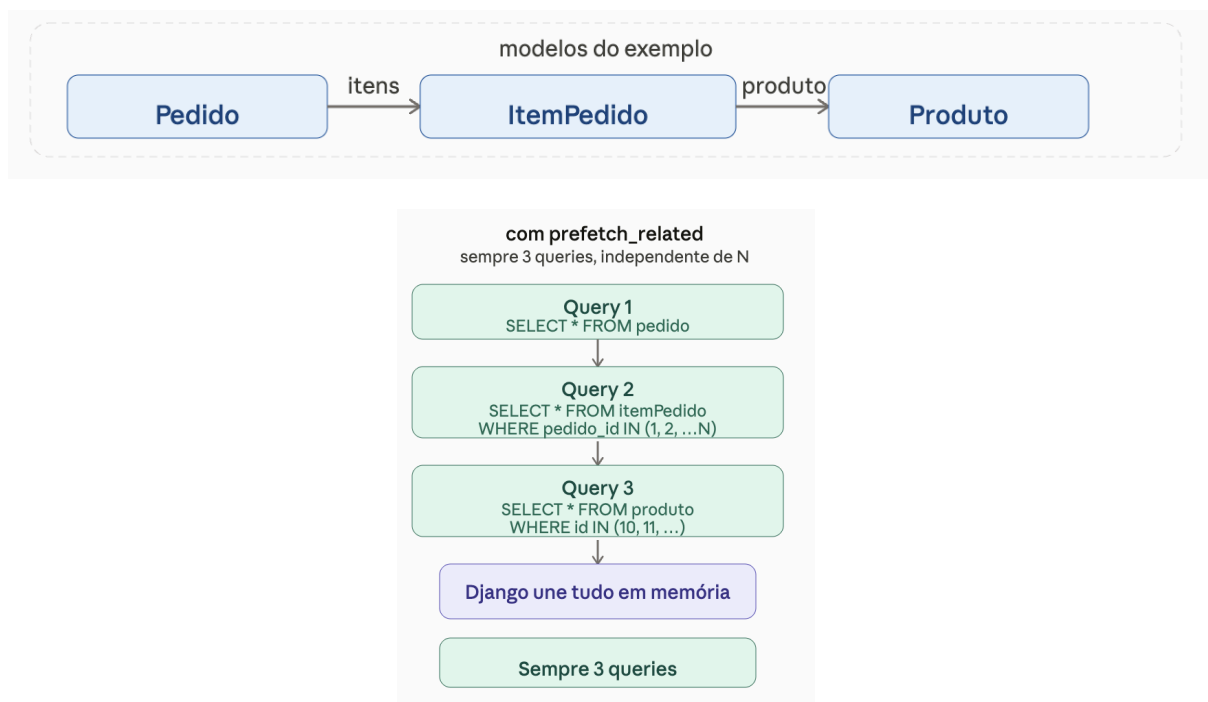


Figura — Exemplo como funciona o prefetch_related

8.2 O Problema N+1 e a Solução

Quando um endpoint lista 100 pedidos e cada serialização acessa `pedido.cliente.nome`, o ORM dispara 1 consulta para listar os pedidos + 100 consultas para buscar cada cliente. Isso é o problema N+1, e ele degrada a performance dramaticamente.

Método	Para que serve	Tipo de Relação
<code>select_related()</code>	Faz JOIN SQL na mesma consulta	ForeignKey, OneToOne
<code>prefetch_related()</code>	Faz consulta separada e junta em memória	ManyToMany, reverso de FK

Tabela 25.1 — `select_related` vs `prefetch_related`

 **Encadeamento:** É possível encadear navegação com `__` no `prefetch_related`, como em `prefetch_related('itens__produto')`. Isso traz, em uma consulta extra, todos os produtos referenciados pelos itens dos pedidos retornados — eliminando completamente o N+1 mesmo em listagens com objetos aninhados.

9. Registrando as Novas Rotas

```
# amazon/urls.py
router = DefaultRouter()
router.register(r'clientes', views.ClienteViewSet, basename='cliente')
router.register(r'vendedores', views.VendedorViewSet, basename='vendedor')
router.register(r'produtos', views.ProdutoViewSet, basename='produto')
router.register(r'perfis-vendedores', views.PerfilVendedorViewSet,
                basename='perfil-vendedor') # NOVO
router.register(r'pedidos', views.PedidoViewSet, basename='pedido') # NOVO
router.register(r'itens-pedido', views.ItemPedidoViewSet,
                basename='item-pedido') # NOVO
```

Código 26.1 — Rotas adicionais em `amazon/urls.py`

Recurso	Endpoint Base	Operações
Pedidos	<code>/amazon_api/pedidos/</code>	GET, POST, PUT, PATCH, DELETE
Itens de Pedido	<code>/amazon_api/itens-pedido/</code>	GET, POST, PUT, PATCH, DELETE
Perfis de Vendedor	<code>/amazon_api/perfis-vendedores/</code>	GET, POST, PUT, PATCH, DELETE

Tabela 26.1 — Endpoints gerados pelos novos ViewSets

10. Testando os Relacionamentos

Para validar que os relacionamentos funcionam de ponta a ponta, vamos criar um pedido completo (com vários itens) em uma única requisição POST. Antes disso, garanta que existam pelo menos um cliente e dois produtos cadastrados, anotando os respectivos IDs.

10.1 POST — Criando um Pedido com Itens Aninhados (Swagger UI)

Em `http://localhost:8000/swagger/`, expanda a seção pedidos, clique em POST `/amazon_api/pedidos/` e em Try it out. Cole o seguinte corpo:

```
{
  "cliente": 1,
  "status": "pendente",
  "observacoes": "Entregar após as 18h",
  "itens": [
    {
      "produto": 1,
      "quantidade": 2,
      "preco_unitario": "349.90"
    },
    {
      "produto": 2,
      "quantidade": 1,
      "preco_unitario": "129.90"
    }
  ]
}
```

JSON 27.1 — Corpo da requisição POST com itens aninhados

Resposta esperada (HTTP 201 Created):

```
{
  "id": 1,
  "cliente": 1,
  "cliente_nome": "Maria da Silva",
  "status": "pendente",
  "data_pedido": "2025-06-10T15:00:00Z",
  "observacoes": "Entregar após as 18h",
  "itens": [
    {
      "id": 1,
      "produto": 1,
      "produto_nome": "Teclado Mecânico RGB",
      "quantidade": 2,
      "preco_unitario": "349.90",
      "subtotal": "699.80"
    },
    {
      "id": 2,
      "produto": 2,
      "produto_nome": "Mouse Sem Fio",
      "quantidade": 1,
      "preco_unitario": "129.90",
      "subtotal": "129.90"
    }
  ],
  "total": "829.70"
}
```

JSON 27.2 — Resposta com itens e total calculados automaticamente

10.2 REST Client — Adicionar ao `api_tests.http`

```
#####
### PEDIDOS (com relacionamentos)
#####

### 17. Listar todos os pedidos com itens aninhados
```

```

GET {{base}}/pedidos/
Accept: application/json

###
### 18. Criar pedido com itens em uma única requisição
POST {{base}}/pedidos/
Content-Type: {{contentType}}

{
  "cliente": 1,
  "status": "pendente",
  "itens": [
    { "produto": 1, "quantidade": 2, "preco_unitario": "349.90" },
    { "produto": 2, "quantidade": 1, "preco_unitario": "129.90" }
  ]
}

###
### 19. Listar pedidos de um cliente específico
GET {{base}}/pedidos/?cliente=1
Accept: application/json

###
### 20. Atualizar status do pedido
PATCH {{base}}/pedidos/1/
Content-Type: {{contentType}}

{
  "status": "pago"
}

```

Arquivo 27.1 — Testes de relacionamento no REST Client

10.3 Insomnia / Postman — Coleção Pedidos

COLEÇÃO: Amazon API → PASTA: Pedidos	
1. GET	Listar Pedidos (com itens aninhados) URL: {{base_url}}/pedidos/
2. POST	Criar Pedido com Itens URL: {{base_url}}/pedidos/ Header: Content-Type: application/json Body: (JSON 27.1)
3. GET	Detalhar Pedido com Itens URL: {{base_url}}/pedidos/1/
4. PATCH	Mudar Status URL: {{base_url}}/pedidos/1/ Body: { "status": "enviado" }
5. DELETE	Cancelar/Remover Pedido URL: {{base_url}}/pedidos/1/

Arquivo 27.2 — Coleção Pedidos no Insomnia/Postman

11. Consultas Reversas no Shell do Django

Uma das maiores vantagens dos relacionamentos é a navegação reversa: a partir de um cliente, listar seus pedidos; a partir de um produto, descobrir quantas vezes ele foi vendido. Vamos explorar essas consultas no shell interativo:

```
python manage.py shell

>>> from backend.models import Cliente, Pedido, Produto, Vendedor

# (1) Listar todos os pedidos de um cliente
>>> cliente = Cliente.objects.get(pk=1)
>>> cliente.pedidos.all()
<QuerySet [(<Pedido: Pedido #1 – Maria da Silva>, ...)]>

# (2) Listar os itens de um pedido
>>> pedido = Pedido.objects.get(pk=1)
>>> pedido.itens.all()
<QuerySet [(<ItemPedido: 2x Teclado Mecânico RGB>, ...)]>
>>> pedido.total
Decimal('829.70')

# (3) Saber em quantos pedidos um produto foi vendido
>>> produto = Produto.objects.get(pk=1)
>>> produto.itens_vendidos.count()
5

# (4) Acessar o perfil de um vendedor (1:1)
>>> vendedor = Vendedor.objects.get(pk=1)
>>> vendedor.perfil.chave_pix
'carlos@loja.com'

# (5) Filtros usando relacionamento (sintaxe __)
>>> Pedido.objects.filter(cliente__nome__icontains='Maria')
>>> Pedido.objects.filter(itens__produto__categoria='eletronicos').distinct()
```

Terminal 28.1 — Consultas reversas no shell do Django

 **Sintaxe __ (lookup):** O duplo underscore navega entre relacionamentos em filtros. cliente__nome significa 'campo nome do objeto cliente relacionado'. Você pode encadear quantos níveis precisar: itens__produto__vendedor__nome.

12. Resumo da Aula

Esta aula consolidou os três tipos de relacionamento do ORM do Django, aplicados ao cenário coeso da Amazon API. O quadro abaixo sintetiza o que cada componente passa a oferecer após esta extensão:

Passo	Componente	Responsabilidade
1	PerfilVendedor (1:1)	Estende o Vendedor com dados fiscais e bancários
2	Pedido (N:1 com Cliente)	Representa uma compra com status e total calculado
3	ItemPedido (N:N com atributos)	Junção entre Pedido e Produto com quantidade e preço

Passo	Componente	Responsabilidade
4	on_delete	Define política ao apagar o registro pai (CASCADE/PROTECT)
5	related_name	Permite navegação reversa (cliente.pedidos.all())
6	Serializers aninhados	Recebem e devolvem objetos relacionados em uma só requisição
7	create() / update() customizados	Trata escrita de objetos aninhados manualmente
8	select_related / prefetch_related	Elimina o problema N+1 nos QuerySets
9	Sintaxe __ em filtros	Filtra por campos de objetos relacionados
10	Testes (Swagger / REST Client / Insomnia)	Validação ponta a ponta dos novos endpoints

Tabela 29.1 — Síntese dos componentes desta aula

12.1 Próximos Passos

- Adicionar o campo vendedor (ForeignKey) ao modelo Produto, criando uma quarta dimensão de relacionamento;
- Implementar permissões para que cada cliente só veja seus próprios pedidos (autenticação JWT);
- Criar actions customizadas no PedidoViewSet, como POST /pedidos/{id}/cancelar/;
- Substituir SerializerMethodField por annotate() do ORM para mover cálculos de total para o banco.

 **Reflexão final:** Modelar relacionamentos é, antes de tudo, modelar regras de negócio. Cada decisão sobre on_delete, sobre permitir ou não null, e sobre serializar ou não objetos aninhados, expressa uma regra do mundo real. Domine essa tradução e você dominará grande parte do trabalho de um analista de sistemas.